

Lecture 13: NoSQL & the JSON Data Model

Amit Kumar Dhar

September 10, 2025

NoSQL Databases

Advantages of NoSQL Databases

NoSQL databases are designed to address the limitations of traditional RDBMS in modern, large-scale applications.

- **High Scalability:** Designed to scale out horizontally across many commodity servers.
- **Flexible Data Models:** Support for unstructured and semi-structured data allows for rapid development and evolution.
- **High Availability:** Architected for distribution and fault tolerance, eliminating single points of failure.
- **Performance:** Optimized for specific data models and access patterns (e.g., fast key-value lookups).

Types of NoSQL Databases

NoSQL is a broad category with several different data models:

- **Key-Value Stores:** Simple hash tables (e.g., **Redis**, **DynamoDB**).
- **Document Stores:** Store structured documents like JSON (e.g., **MongoDB**, **Couchbase**).
- **Graph Databases:** Model data in terms of nodes and relationships (e.g., **Neo4j**, **Amazon Neptune**).
- **Vector Databases:** Designed to store and query high-dimensional vectors for AI/ML applications (e.g., **Pinecone**, **Weaviate**).
- **Column-Family Stores:** Store data in columns rather than rows, good for analytical queries (e.g., **Cassandra**, **HBase**).

The JSON Data Model

Why JSON for Document Stores?

JSON (JavaScript Object Notation) has become the de facto standard for document databases.

Advantages:

- **Human-Readable:** The syntax is lightweight and easy to read/write.
- **Language Independent:** Natively supported by nearly every programming language.
- **Maps to Objects:** The structure of a JSON document maps directly to objects in object-oriented programming.
- **Flexible & Hierarchical:** Supports nested structures (objects within objects, arrays of objects), allowing related data to be stored together in a single document.

JSON Overview & Example

Definition: JSON is a lightweight data-interchange format that uses key-value pairs and ordered lists.

Example JSON Document:

```
{  
  "sid": 101,  
  "sname": "Alice",  
  "major": "CS",  
  "isActive": true,  
  "gpa": 3.9,  
  "courses": [  
    {"cid": "CSL303", "grade": "A"},  
    {"cid": "CSL211", "grade": "A"}  
  ]  
}
```

JSON vs. Relational Data Model

Feature	Relational Model	JSON/Semi-structured
Schema	Schema-on-Write (Structure enforced)	Schema-on-Read (Structure is flexible)
Structure	Flat tables (rows/cols)	Hierarchical (nested)
Relationships	Foreign Keys, Joins	Embedding, Linking
Normalization	Encouraged	Denormalization is common

JSON Terminologies

- **Name-Value Pair:** The basic building block. A name (string) followed by a value.

```
"sname": "Alice"
```

- **Object:** An unordered collection of name-value pairs, enclosed in curly braces {}.

```
{ "sid": 101, "sname": "Alice" }
```

- **List (Array):** An ordered collection of values, enclosed in square brackets [].

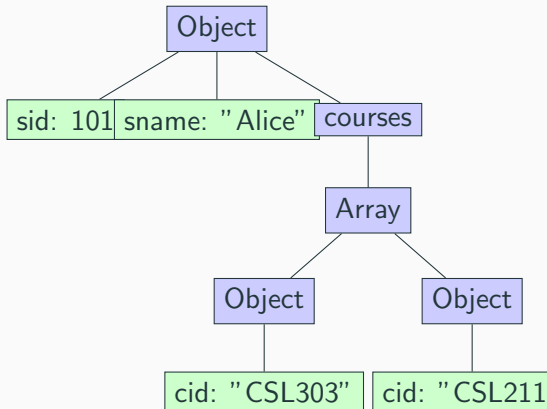
```
[ "CS", "EE", "ME" ]
```

JSON Data Types: String, Number, Boolean (true/false), Object, Array, and null.

JSON Semantics as a Tree

A JSON document has a natural tree structure, where objects and arrays are internal nodes and their values are children.

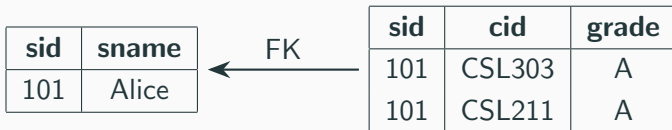
```
{  
  "sid": 101,  
  "sname": "Alice",  
  "courses": [  
    {"cid": "CSL303"},  
    {"cid": "CSL211"}  
  ]  
}
```



Mapping Relational to JSON

Example 1: One-to-Many (Student & Enrollments)

In a relational model, a student's enrollments are stored in a separate table, linked by a foreign key.



In JSON, we can **embed** the enrollments inside the student document.

```
{ "sid": 101,  
  "sname": "Alice",  
  "enrollments": [  
    {"cid": "CSL303", "grade": "A"},  
    {"cid": "CSL211", "grade": "A"} ]  
}
```

Example 2: Many-to-Many (Students & Courses)

A many-to-many relationship requires a join table (Enrolled).



In JSON, we can embed the course information directly, removing the need for a "join" during reads.

```
{ "sid": 101,  
  "sname": "Alice",  
  "courses_taken": [  
    {"cid": "CSL303", "cname": "Databases", "grade": "A"},  
    {"cid": "CSL211", "cname": "Data Structures",  
     "grade": "A"}  ]  
}
```

Thank You!
Questions?